2023 IEEE International Conference on Quantum Software (QSW) | 979-8-3503-0479-4/23/$31.00 ©2023 IEEE | DOI: 10.1109/QSW59989.2023.00016

A Testing Pipeline for Quantum Computing Applications

Colin Kai-Uwe Becker, Ilie-Daniel Gheorghe-Pop, Nikolay Tcholtchev
Fraunhofer Institute for Open Communication Systems (FOKUS)
{firstname.lastname}@fokus.fraunhofer.de

***Abstract*—As the scientific community is increasing the coverage of the quantum computing domain ranging from theoretical physics to financial applications, so does the number of quantum computing applications being developed for quantum computing hardware that is available today. As this hardware evolves towards increased capacity in terms of qubit number and increased fidelity, so does the complexity of quantum computing applications that implement the state-of-the-art of (hybrid) quantum algorithms. With this comes an increased demand for quantum software engineering practices where testing is a crucial part of the development process. Hence, a robust approach that addresses certain challenges and peculiarities related to the quantum computing domain is needed for the testing process. This paper proposes a standard-ready testing pipeline that covers the requirements of testing in a continuous development cycle as part of a modern quantum software engineering lifecycle.**

***Index Terms*—Quantum computing, Quantum Software Engineering, NISQ, Testing, Debugging**

I. Introduction

In the current *NISQ* (Noisy Intermediate-Scale Quantum) era of quantum computing, the level of excitement in the scientific community remains ever-increasing as shown in the yearly publication numbers [1], despite the current limitations of available quantum computing hardware in terms of result accuracy, consistency, scale and overall experimentation cohesion across different providers. A major challenge in this race towards practical applications and the undisputed quantum advantage remains the structured development of quantum computing applications on a larger scale and complexity. Therein, experimentation on currently available *quantum processing units* (QPUs) plays a major role for the planning and design of the application as well as for testing and deployment. In the following, we present our approach to address this challenge by introducing a testing pipeline for testing quantum computing applications within a modern quantum software engineering lifecycle.

Based on the commonly adapted concept of software development lifecycles, extended concepts that include the area of quantum computing were presented in the recent years [2], [3]. The integrated development and operation of quantum computing applications thereby is dealt with similar to how the IT industry deals with classical computing applications. Since hybrid quantum-classical algorithms as of today promise computational advantage potential for real-world applications, it is necessary that the classical software development is compatible with software engineering methods used for quantum computing applications as well, thereby raising the requirement for a seamless co-development of classical and quantum computing code. In this work, we will focus on the development subcycle of the quantum software engineering lifecycle that is shown in Figure (1), i.e., the processes from Plan to Test. More specifically, we propose a testing pipeline for the Test phase which is crucial for *continuous delivery*. The starting point for initiating the testing phase is a quantum computing application that was developed according to a prior created requirements and design document (Plan), implemented with a quantum computing SDK of choice (Code) and compiled such that it is able to run on different hardware backends using their respective APIs and firmware (Build). The transpilation to the hardware characteristics of available target QPUs is not necessary at this stage. The Test phase which is described in detail in the next section then leads to the Release phase in which the application is packaged and versioned for the following deployment.

We aim to provide a standard-like sequence of processes that cover the testing requirements of such a quantum software engineering lifecycle. In addition, we provide insights into belonging challenges and solutions based on which the development of suitable tools assisting the proposed processes can be pursued.

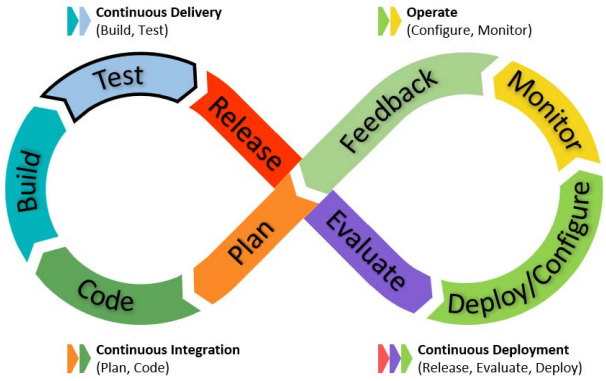

Fig. 1: Quantum DevOps lifecycle as described in [2]. The Test phase relevant for this article is highlighted.

979-8-3503-0479-4/23/$31.00 ©2023 IEEE
DOI 10.1109/QSW59989.2023.00016

Authorized licensed use limited to: CBS Library. Downloaded on March 31,2026 at 21:13:08 UTC from IEEE Xplore. Restrictions apply.

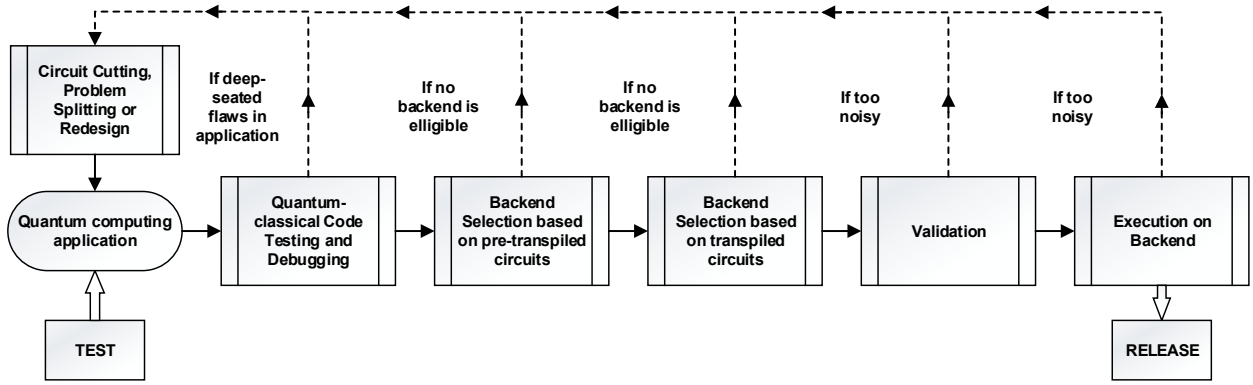

Fig. 2: Diagrammatic image of the proposed testing pipeline. The input is a quantum computing application that was developed in the CODE and BUILD phase and now undergoes the TEST phase. Before entering the RELEASE phase, the application needs to be tested for validity, be thoroughly inspected for bugs, eventually be split into smaller parts or redesigned and finally be transpiled to a suitable QPU backend so that test data under the influence of realistic quantum noise can be gathered, evaluated and verified. With successful tests, the application is ready for the operations subcycle.

II. TESTING PIPELINE

In this main section, we describe the activities that need to be performed in the TEST phase of a quantum software engineering lifecycle as a part of an (automated) testing pipeline that caters for the needs of testing quantum computing software such as code testing and debugging as well as resource estimation and result verification on specifically chosen NISQ devices before the RELEASE phase. Thereby, we focus on quantum computing applications that target gate-based QPUs.

A. Code Testing & Debugging

At the beginning of the proposed testing pipeline stands the initial testing and debugging of the source code of the quantum computing application. This application may include, but is not limited to, classical preprocessing, the generation of the explicit quantum circuits for various input parameters and problem sizes, the coordination, monitoring and execution of multiple iterations in a hybrid scenario consisting of classical and quantum algorithms or subroutines, as well as the interpretation of the results via post processing of the retrieved data. Testing and debugging the source code for the classical subroutines and post processing steps is thereby well researched, whereas similar concepts for the quantum computing code are still relatively sparsely studied. The current research, among others topics, includes developing a taxonomy of quantum computing bugs [4] and concepts for quantum circuit testing and debugging tools [5], [6], [7], formal language verification tools [8] as well as runtime assertions for quantum circuits [9]. In the following, some crucial challenges for testing and debugging quantum computing code are being listed:

1) **State monitoring:** During the execution of a quantum computing application on a QPU, the quantum state in the system cannot be monitored without performing destructive measurements. Furthermore, due to the *no-cloning theorem*, a quantum state cannot be copied, nor, for this very reason, is technology such as *QRAM* available. Quantum state monitoring in a limited scope is used in *quantum error correction codes* where errors during the execution of a quantum circuit can be detected via mid-circuit *syndrome measurements* and conditionally corrected dynamically afterwards. However, as of today, performing error correction effectively neither is possible for useful circuit sizes on NISQ devices, nor can it be applied to find bugs in the actual implementations.

2) **Performing classical simulations:** For testing quantum computing applications, we often need to rely on the use of classical simulators. These allow for monitoring the quantum state and finding errors in the implementation, however, such simulations are computationally expensive and in practice these resources are severely limited, which leads to applicability only for small problem sizes or for circuits that are efficiently classically simulatable.

3) **Verification:** Verifying the output from a quantum computing application that was performed on a QPU for problem sizes that are not feasible for being tested via classical simulation can be challenging. For example, a quantum simulation of a physical system where there is no known analytical solution or classical simulation to compare the quantum simulation to, it is difficult to verify the correctness of the output and hence also ensuring that the quantum computing application was implemented correctly for such problem sizes. There is ongoing research about verification protocols on NISQ devices [10].

Based on these challenges, while not optimal, in practice the usage of classical simulators for testing the correctness of arbitrary quantum computing applications is mandatory

Authorized licensed use limited to: CBS Library. Downloaded on March 31,2026 at 21:13:08 UTC from IEEE Xplore. Restrictions apply.

but can often only be used for smaller problem sizes or certain subroutines of the applications, e.g., state or probability distribution initialization and pre-measurement basis transformation and algorithmic subroutines such as *quantum fourier transform* (QFT) or *Grover diffusion operators*. Hence, the correct scalability can only be assumed inductively based on the collected data without running the applications on real QPUs, which also does not guarantee verifiable test results. In the following, we discuss some further requirements for testing quantum computing applications.

Unit tests are an essential part of software development that involve testing individual functions or modules of code to ensure they are working correctly and hence help to identify bugs and errors in the code early on in the development cycle, which can save time and resources. This also holds true for quantum software development, especially since quantum computing code can be difficult to write and even more challenging to debug as was discussed above. Furthermore, unit tests can help to ensure that individual quantum circuits and functions are working correctly, reducing the risk of introducing errors into the system. Unit tests can help to validate the correctness of the output of a quantum circuit under various noise models and conditions, which is important for the development of reliable quantum algorithms and applications. For this, an easy-to-use and versatile syntax for writing unit tests that cater for the peculiarities of testing quantum computing applications is required. It is worth noting that in *Q#*, unit testing functionality is already implemented. Unfortunately, this does not apply to most of the known frameworks that use the *Python* language.
In the following, we record several *assertion* types requiring as input a quantum circuit, a simulation or QPU backend and certain user-specified error rates. This aims to simplify and assist the implementation of unit tests:

`AssertProbabilityEquality`: This assertion returns `True` if the output probability distribution Q defined by the results of a (noisy) simulation or the execution of a circuit on a QPU is near the theoretically predicted probability distribution P. Otherwise it returns `False`. This can, for example, be evaluated with the *squared Hellinger fidelity*

$$\mathcal{F}(P,Q) = (\sum_i \sqrt{p_i q_i})^2 \quad \text{with} \quad \mathcal{F}(P,Q) \in [0,1]$$

by setting a user-specified fidelity threshold $c \in [0,1]$ such that `True` is returned only if $\mathcal{F}(P,Q) \geq c$. This is especially useful for probing if noisy results are in a user-specified acceptance range which is a major criteria for the validation process described in Section (II-C).

`AssertExpectationValueEquality`: This assertion returns `True` if the sampled expectation value $\langle \mathcal{O} \rangle$ for a measured observable $\mathcal{O}$ is near the theoretically predicted expectation value up to a user-specified deviation. Otherwise, it returns `False`. Similar as above, this is particularly useful for the validation process.

`AssertUnitaryEquality`: This assertion returns `True` if the simulated unitary A is equivalent up to a user-specified relative error $\epsilon_{\text{relative}}$ and absolute error $\epsilon_{\text{absolute}}$ to the theoretically predicted unitary B. This means that the following equation holds element-wise (granted that both matrices have the same dimension):

$$|a_{ij} - b_{ij}| \leq (\epsilon_{\text{absolute}} + \epsilon_{\text{relative}} * |b_{ij}|).$$

Otherwise, it returns `False`. A typical use case for this is testing if a quantum circuit, its subroutines or specific user-defined gates are implemented correctly, hence it suffices to only cater for rounding errors in the numeric values.

`AssertStatevectorEquality`: This assertion returns `True` if the simulated state vector $|\phi\rangle$ corresponds to the theoretically predicted statevector $|\psi\rangle$ up to a user-specified relative error $\epsilon_{\text{relative}}$ and absolute error $\epsilon_{\text{absolute}}$. Similar as for asserting the equality of unitary matrices above, the following equation needs to hold element-wise:

$$|\phi_i - \psi_i| \leq (\epsilon_{\text{absolute}} + \epsilon_{\text{relative}} * |\psi_i|).$$

Otherwise, it returns `False`. The usage is analogous to `AssertUnitaryEquality`.

`AssertDistributionStability`: This assertion returns `True` if the resulting output probability distribution does not change significantly regarding the number of shots for sampling the distribution according to a user-specified error. Otherwise, it returns `False`. The implementation will use `AssertProbabilityEquality` while the number of shots is scaled incrementally. A use case for this is fine tuning the number of shots in noisy simulations so that this value can be used for testing and running the application on a real QPU. Similar to this, one can also use an assertion for testing the stability of expectation values regarding the number of shots per circuit execution.

When writing test functions that - not exclusively - use specially defined assertion types, a simulation or a QPU backend needs to be passed. A comprehensive and easy to use syntax for implementing this in *Python* that is inspired by the unit test implementation in *Q#* is shown in the code example below. Here we make use of *decorators* to pass a pre-configured noisy backend object to the example test function. Hence, it is not needed to be passed at function execution. The shown assertion is an implementation of `AssertProbabilityEquality` introduced above.

```
@NoisyQPU(number_of_qubits = 5)
def test_example(error_tolerance, shots, backend):

    qc = initialize_bell_circuit(n = 2)
    optimal_res = {'00': 0.5, '11': 0.5}

    return assert_probability_equality
           (qc, optimal_res, error_tolerance, shots,
            backend)

#The test case evaluates to True after the Hellinger
#fidelity is computed between the simulated and
#expected distribution.
test_example(error_tolerance = 0.98, shots = 3000)
```


Authorized licensed use limited to: CBS Library. Downloaded on March 31,2026 at 21:13:08 UTC from IEEE Xplore. Restrictions apply.

In the testing and debugging phase, however, it is usually not necessary to test with noisy simulations as the goal of this process is to ensure the correctness of the code rather than the noise-robustness which is handled later in the proposed pipeline. Language specific features such as automatic *uncomputation* and qubit management as for example proposed in [11] can further assist debugging and testing of quantum computing applications.

B. Backend Decision

Deciding for suitable QPU backends that enable the execution of given quantum computing applications with high success probabilities is crucial for testing and deploying newly developed quantum computing applications as well as running quantum computing experiments in general. Such a backend decision process is implemented in the proposed testing pipeline via two successive subprocesses. At first, we filter for eligible backends based on the quantum computing application that is not yet mapped to the specifications of any NISQ device and then based on the reduced set of backends a *resource estimation* for the transpiled application is performed which leads to a final decision for the best available backend(s). Starting with the transpiled quantum circuits and considering all available QPU backends at once comes with large computational costs and should therefore be avoided for efficient testing.
For the first QPU backend filtering, the available backends are filtered based on the following criteria (this is very similar to the approach described in [12]):

1) **User account:** With known cloud quantum computing providers, detailed information about the current status of their backends, such as detailed error rates after the last calibration, is not guaranteed to be available via the respective APIs without a paid ticket or token. Therefore, instead of considering all available cloud backends and providing the user information about which backend would suit their need best (if any at all), independent of access-policies, we consider the case that the user already has access to a collection of backends. Therefore, only this reduced set of backends is supported because the availability of detailed error metrics is crucial for the following decision criteria.

2) **Feature compatibility:** The backends are further filtered based on the used features in the provided quantum computing application, such that it can be assured that the backends support these features. Prominent examples are the support of dynamic circuits, i.e., mid-circuit measurements and classically conditioned feed-forward operations using values produced by prior measurements to determine what gates to apply next, as well as the availability of hybrid quantum circuit runtimes with the required functional scope and specific available native gates, such as for example *GMS* gates, that would be too costly to approximate on other gate sets if such gates are used frequently in the application.

3) **Circuit width:** All generated circuits of the quantum computing application that should be tested for a desired problem size are analyzed for their belonging circuit widths. Afterwards, every backend that is not able to fit this number of qubits is filtered out.

For the second round of backend filtering, the quantum computing application at hand is transpiled to the specifications of the backends that remain and then we trigger the *resource estimation process* in order to find the most suitable of these backends. For this, the following criteria are being used:

1) **Circuit cost metric:** A circuit cost metric is frequently used in quantum circuit optimization tools, such that, e.g, the best qubit mappings can be found [13], and is appropriately also used to evaluate and compare quantum circuit optimization tools [14]. An exemplary cost metric, that also suits a resource estimation process is thereby given in [14] via

$$\mathcal{C} = -D \log K - \sum_i \log F_i^{1\mathrm{q}} - \sum_j \log F_j^{2\mathrm{q}},$$

where D is the circuit depth, K a phenomenologically defined penalty factor, $F_i^{1\mathrm{q}}$ the error rate for the i-th gate in the native single qubit gate set and $F_j^{2\mathrm{q}}$ the error rate for the j-th gate in the native two qubit gate set. The above cost function is valid under the assumption that the total error in the circuit is defined by the fidelity of a sequence of quantum gates that can be factorized in the product of individual gates fidelities. Hence it serves only as a rough estimator that can be further refined by considering additional error metrics such as crosstalk errors if the data for the target backends are available.

2) **Active qubit time and coherence time:** The active qubit time for a single shot can be approximated based on the gate timings defined in the pulse schedules for the transpiled circuits. Let t^n_{start} denote the clock time at which the n-th qubit is first active, i.e., performs the first scheduled pulse and t^n_{end} the respective clock time at which the last pulse is scheduled for the n-th qubit. Then the approximated longest active qubit time Δt is given by

$$\Delta t_{\max} = \max(t^0_{\mathrm{end}} \ldots, t^n_{\mathrm{end}}) - \min(t^0_{\mathrm{start}}, \ldots, t^n_{\mathrm{start}}).$$

This can be compared to the median coherence times T_1 and T_2 of the backend to infer how large the noise impact of decoherence effects will be. For a more in-depth analysis, individual qubit times $\Delta t_n = t^n_{\mathrm{end}} - t^n_{\mathrm{start}}$ can be compared to the qubit-specific coherence times. Note that this quickly gets unwieldy with an increasing number of qubits. This analysis is connected to the phenomenologically defined penalty factor K in the cost metric above that defines a weight for the cost

Authorized licensed use limited to: CBS Library. Downloaded on March 31,2026 at 21:13:08 UTC from IEEE Xplore. Restrictions apply.

introduced by the circuit depth parameter.

3) **Additional benchmark metrics:** As device specific quantum noise cannot be modeled precisely but only approximated [15], metrics such as the circuit cost estimation described above will persistently have their limits. For this, it can be useful to employ additional benchmark metrics that are geared towards application-centric performance predictions. Examples for this can be performance benchmarks of circuit subroutines similar to those identified in the testing and debugging phase that can be executed on NISQ devices. A blueprint for a performance prediction methodology where such circuit subroutines can be implemented and evaluated in a *volumetric benchmarking framework* can be found in [16]. The *mirror benchmarking framework* [17] potentially can be used for easier result verification while maintaining similar experiment sizes as with the standard volumetric benchmarking approach, however a general implementation for arbitrary native gate sets was not yet published.

If several backends with similar resource estimation results are found, we advise to finally filter for the least busy backend for the initial tests in order to get the results as fast as possible. Some of the remaining backends could be used for obtaining results for comparisons and all backends that survived the above filtering process should be stored in the requirements document for the developed quantum computing application which is important for ongoing tests, evaluations and later deployment. This list of backends should be updated in certain intervals, since the error metrics and coherence times will change depending on the system calibration quality, hence also the (noise-aware) transpilation results and the evaluation of a circuit cost metric and benchmarking results. If in either of the two backend decision steps no eligible backend is found, then using a splitting method or having a redesign of the application such us described in (II-D) below is advised. Continuing with the following validation or tests on a real QPU then has a higher probability for unsatisfactory results that are not on par with the output requirements of the developed application. However, as things stand at present, neither a successful backend decision process (and follow-up validation process) nor unsuccessful results in these processes can be guaranteed with full confidence leading to a particular output quality of the application when tested on a real QPU.

C. Validation

Further testing the transpiled quantum computing application for the chosen backends through simulations under the influence of different noise-models provides valuable insights into the expected performance when finally running the application on a noisy QPU. For device specific noise models, such as present in the *Qiskit* framework, in many cases, simulations generate optimistic results that do not directly correspond to the noisy outputs of real quantum computing hardware. Therefore, if the validation experiments fail, this can be a good indicator to proceed with the steps in (II-D). The developed and transpiled application can also be probed with non device-specific noise models and error sources for noise-robustness. The obtained results from the validation process hence also provide key information for further optimizations and backend decision processes in subsequent development iterations.

D. Redesign, Problem splitting & Quantum Circuit Cutting

While the steps in the proposed testing pipeline are run through one after the other, there is a possibility that a redesign of the quantum computing application is necessary if in the testing and debugging phase, deep-seated errors become apparent that cannot be easily fixed. Furthermore, if in either one of the backend decision steps, no eligible backend is found, a splitting approach such as separating the problem definition in smaller subproblems (*divide and conquer approach*) or cutting the quantum circuits belonging to the quantum computing application in smaller pieces via *Quantum circuit cutting* methods can be pursued. The same goes for the validation step and the execution on the decided QPU backend; if the results do not satisfy the requirements, those methods can also be an option. Suitable *quantum error mitigation (QEM)* methods [18] can furthermore be used to improve noisy results.

Quantum circuit cutting is an approach for running quantum circuits that exceed the circuit size limitations imposed by current NISQ devices and hence are physically not compatible with said devices. This is achieved by finding suitable locations in the quantum circuits respectively, where the circuits can be cut according to user-defined specifications, resulting in smaller sub-circuits that can potentially be executed on the target backends. Such cuts can either be found in a way to reduce the number of qubits (*qubit-cutting*) or to reduce the circuit depth (*wire-cutting*). There are different approaches for finding those cuts, for example, in *CutQC* [19] a mixed integer programming cut searcher with the *Gurobi* optimizer [20] is used to find optimal cut locations, whereas the randomized measurement circuit cutting approach [21] implemented in *pennylane* [22] employs the hypergraph partitioning library *KaHyPar* [23] as a cut searcher. Naturally, this introduces relevant computational overhead with exponential scaling in the number of qubits and cuts. Moreover, running multiple smaller sub-circuits instead of the initial larger circuit introduces a *sampling overhead* [24] and is therefore much less efficient which is also valid for the problem splitting approach mentioned above. Finally, it is necessary to further process the obtained results from each sub-circuit in order to recreate the probability distribution of the initial circuit. We therefore consider using these methods only as a backup in order to be able to test the developed application for problem sizes that exceed available classical simulation methods and QPU backend capabilities. It can also provide useful information about requirements that the hardware needs to fulfill to run the initial application.

E. Execution on noisy QPU

After a set of suitable QPU backends have been decided on and the validation process have proven the quantum computing application according to required acceptance criteria,

Authorized licensed use limited to: CBS Library. Downloaded on March 31,2026 at 21:13:08 UTC from IEEE Xplore. Restrictions apply.

the application can finally run on a real QPU with the same set of inputs and problem sizes. If the developed application already passed multiple iterations in the software engineering lifecycle, further tests such as testing the execution speed or quality metrics, e.g., reliability, reproducibility and security yields relevant indicators for identifying the need for future changes, improvements and extensions in the following lifecycle iterations. Otherwise, similar as for the previous steps in the testing pipeline, the application can undergo further redesigns, the problem statement potentially can be split or the circuit size be directly reduced via quantum circuit cutting methods.

III. Conclusion

This paper has provided first insights into a testing pipeline for quantum computing applications following a sequence of different processes such as unit testing and debugging, backend selection, validation with noisy simulations as well as performing tests on real QPUs. Furthermore, recommendations for when problem splitting, quantum circuit cutting or redesigns of the developed application should be performed were presented. The proposed testing pipeline thereby demonstrates a comprehensive approach for the activities that need be performed in the TEST phase within a quantum software engineering lifecycle. This robust approach to testing is supporting practical experimentation in the NISQ era and provides a standard-ready framework for which different tools supporting the requirements of the individual processes can be developed. For future work, the proposed testing pipeline will be demonstrated with several quantum computing use cases. Moreover, a testing framework that caters for debugging and unit testing functionality will be developed in the *Python* language. This will cover functionalities as well as quality-of-life improvements that make debugging as well as writing and running unit tests more streamlined and accessible for software developers with a classical computing background.

Acknowledgement

This work was funded by the Federal Ministry for Economic Affairs and Climate Action (German: Bundesministerium für Wirtschaft und Klimaschutz) under the project funding number 01MQ22007A. The authors are responsible for the content of this publication.

References

[1] Elsevier, "Quantum computing research trends report," 2022, accessed on March 1st, 2023. [Online]. Available: https://www.elsevier.com/solutions/scopus/research-and-development/quantum-computing-report

[2] I.-D. Gheorghe-Pop, N. Tcholtchev, T. Ritter, and M. Hauswirth, "Quantum devops: towards reliable and applicable nisq quantum computing," in *2020 IEEE Globecom Workshops (GC Wkshps*. IEEE, 2020, pp. 1–6.

[3] B. Weder, J. Barzen, F. Leymann, and D. Vietz, "Quantum software development lifecycle," 2021. [Online]. Available: https://arxiv.org/abs/2106.09323

[4] Y. Huang and M. Martonosi, "QDB: From Quantum Algorithms Towards Correct Quantum Programs," in *9th Workshop on Evaluation and Usability of Programming Languages and Tools (PLATEAU 2018)*, ser. OpenAccess Series in Informatics (OASIcs), T. Barik, J. Sunshine, and S. Chasins, Eds., vol. 67. Dagstuhl, Germany: Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik, 2019, pp. 4:1–4:14. [Online]. Available: http://drops.dagstuhl.de/opus/volltexte/2019/10196

[5] S. A. Metwalli and R. Van Meter, "A tool for debugging quantum circuits," 2022. [Online]. Available: https://arxiv.org/abs/2205.01899

[6] A. Miranskyy, L. Zhang, and J. Doliskani, "On testing and debugging quantum software," 2021. [Online]. Available: https://arxiv.org/abs/2103.09172

[7] F. A. Calderon-Vargas, T. Proctor, K. Rudinger, and M. Sarovar, "Quantum circuit debugging and sensitivity analysis via local inversions," *Quantum*, vol. 7, p. 921, Feb. 2023. [Online]. Available: https://doi.org/10.22331/q-2023-02-09-921

[8] L. Zhou, G. Barthe, P.-Y. Strub, J. Liu, and M. Ying, "Coqq: Foundational verification of quantum programs," *Proc. ACM Program. Lang.*, vol. 7, no. POPL, jan 2023. [Online]. Available: https://doi.org/10.1145/3571222

[9] J. Liu and H. Zhou, "Systematic approaches for precise and approximate quantum state runtime assertion," in *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2021, pp. 179–193.

[10] A. Gheorghiu, T. Kapourniotis, and E. Kashefi, "Verification of quantum computation: An overview of existing approaches," *Theory of Computing Systems*, vol. 63, no. 4, pp. 715–808, 2019. [Online]. Available: https://doi.org/10.1007/s00224-018-9872-3

[11] R. Seidel, S. Bock, N. Tcholtchev, and M. Hauswirth, "Qrisp: A framework for compilable high-level programming of gate-based quantum computers," ICFP 2022, 2022. [Online]. Available: https://icfp22.sigplan.org/details/planqc-2022/6/Qrisp-A-Framework-for-Compilable-High-Level-Programming-of-Gate-Based-Quantum-Comput

[12] M. Salm, J. Barzen, U. Breitenbücher, F. Leymann, B. Weder, and K. Wild, "The NISQ Analyzer: Automating the Selection of Quantum Computers for Quantum Algorithms," in *Proceedings of the 14th Symposium and Summer School on Service-Oriented Computing (SummerSOC 2020)*. Springer International Publishing, Dec. 2020, pp. 66–85.

[13] P. D. Nation and M. Treinish, "Suppressing quantum circuit errors due to system variability," 2022. [Online]. Available: https://arxiv.org/abs/2209.15512

[14] Y. Kharkov, A. Ivanova, E. Mikhantiev, and A. Kotelnikov, "Arline benchmarks: Automated benchmarking platform for quantum compilers," 2022. [Online]. Available: https://arxiv.org/abs/2202.14025

[15] K. Georgopoulos, C. Emary, and P. Zuliani, "Modeling and simulating the noisy behavior of near-term quantum computers," *Physical Review A*, vol. 104, no. 6, dec 2021. [Online]. Available: https://doi.org/10.1103%2Fphysreva.104.062432

[16] T. Lubinski, S. Johri, P. Varosy, J. Coleman, L. Zhao, J. Necaise, C. H. Baldwin, K. Mayer, and T. Proctor, "Application-oriented performance benchmarks for quantum computing," 2021. [Online]. Available: https://arxiv.org/abs/2110.03137

[17] T. Proctor, K. Rudinger, K. Young, E. Nielsen, and R. Blume-Kohout, "Measuring the capabilities of quantum computers," *Nature Physics*, vol. 18, no. 1, pp. 75–79, Jan 2022. [Online]. Available: https://doi.org/10.1038/s41567-021-01409-7

[18] Z. Cai, R. Babbush, S. C. Benjamin, S. Endo, W. J. Huggins, Y. Li, J. R. McClean, and T. E. O'Brien, "Quantum error mitigation," 2022.

[19] W. Tang, T. Tomesh, M. Suchara, J. Larson, and M. Martonosi, "Cutqc: Using small quantum computers for large quantum circuit evaluations," in *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, ser. ASPLOS '21. New York, NY, USA: Association for Computing Machinery, 2021, p. 473–486. [Online]. Available: https://doi.org/10.1145/3445814.3446758

[20] Gurobi Optimization, LLC, "Gurobi Optimizer Reference Manual," 2023. [Online]. Available: https://www.gurobi.com

[21] A. Lowe, M. Medvidović, A. Hayes, L. J. O'Riordan, T. R. Bromley, J. M. Arrazola, and N. Killoran, "Fast quantum circuit cutting with randomized measurements," 2022. [Online]. Available: https://arxiv.org/abs/2207.14734

[22] V. Bergholm, J. Izaac *et al.*, "Pennylane: Automatic differentiation of hybrid quantum-classical computations," 2018. [Online]. Available: https://arxiv.org/abs/1811.04968

[23] S. Schlag, "High-quality hypergraph partitioning," Ph.D. dissertation, Karlsruhe Institute of Technology, Germany, 2020.

[24] L. Brenner, C. Piveteau, and D. Sutter, "Optimal wire cutting with classical communication," 2023. [Online]. Available: https://arxiv.org/abs/2302.03366

Authorized licensed use limited to: CBS Library. Downloaded on March 31,2026 at 21:13:08 UTC from IEEE Xplore. Restrictions apply.